CS6650 ASSIGNMENT 2

Message Distribution and the Real Leaderboard

Lin Cai / November 13, 2026

The system accepts ski swipes on AWS, aggregates vertical totals in memory, and pushes real resort leaderboards to WebSocket subscribers across multiple ingest hosts.

Source repository

https://github.com/LinCai-prog/CS6650-Assignment2. The benchmark evidence index contains the logs, exact counters, plots, and supporting experiments. The in-memory implementation was introduced in commit 7d25ac3.

Repository path	Responsibility
/server-v2	HTTP ingestion, lift lookup, pooled AMQP publishing and live WebSockets
/consumer	Ordered aggregation, deduplication, versioned standings and snapshots
/deployment	ALB configuration, backplane setup and deployment
/monitoring	Broker and host samplers and guarded load client
AI_USAGE.md	AI usage reflection with accepted and corrected suggestions

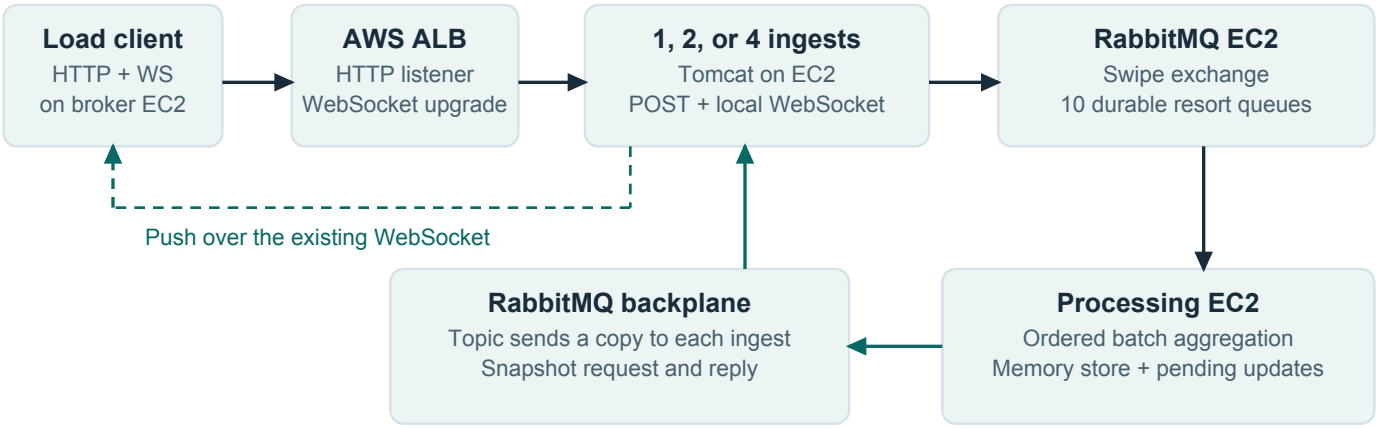
Results

All four required AWS workloads completed. Four ingests processed **1,000,000 swipes** with zero failed requests, broker redeliveries or new dead letters. Its unpaced main phase averaged **2,883.09 requests/s**. The RabbitMQ ACK count, unique Consumer count and successful ingest POST count each matched the workload.

Section	Pages
Architecture and delivery guarantees	2–3
AWS client, queue and EC2 results	4–6
WebSocket correctness, ALB behavior and tuning	7–8
RabbitMQ and CloudWatch console evidence	9–11
Client console excerpts	12

The AWS suite was measured with in-memory aggregation. Earlier SQLite measurements and local tuning runs are identified separately.

Architecture and queue topology



Blue-gray: swipe path. Teal: leaderboard path.

Why a broadcast backplane

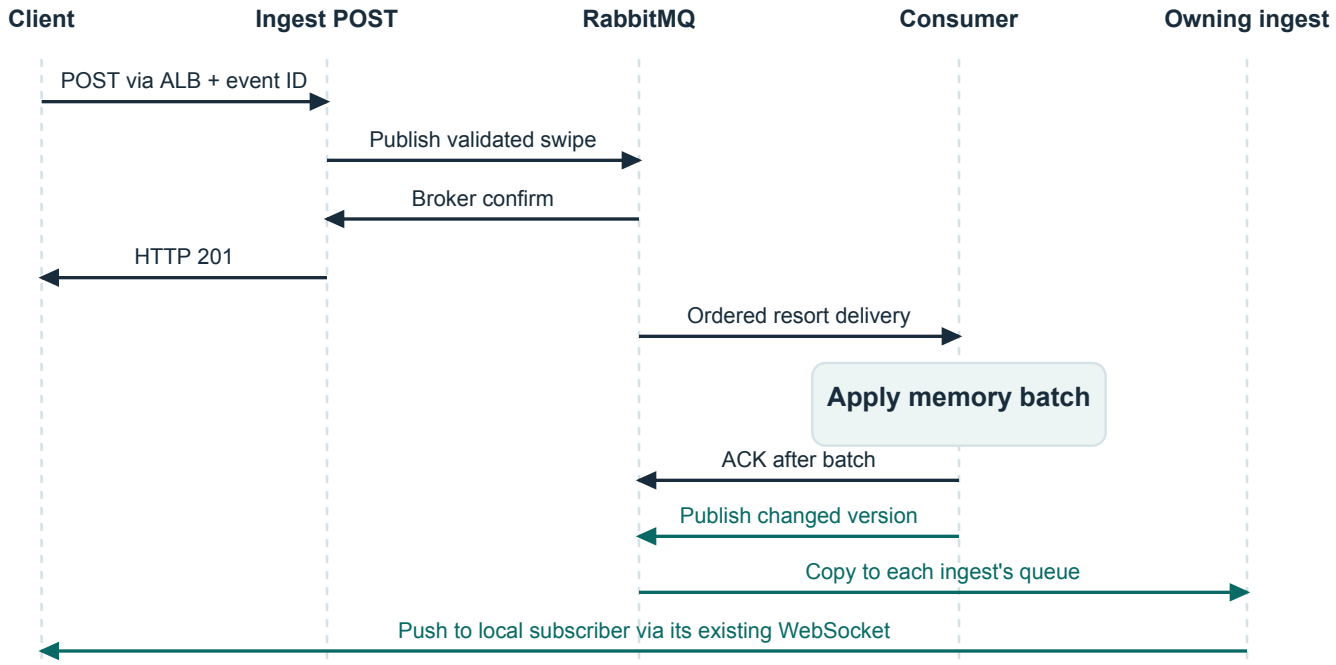
The WebSocket lives on the ingest host that accepted the upgrade. The processing host publishes each changed leaderboard through RabbitMQ. Every ingest gets its own copy and sends it to its local subscribers for that resort and day. A new subscriber asks the processing host for an authoritative snapshot; the ingest applies newer versions as they arrive.

This keeps HTTP target selection independent of resort processing. It costs an extra broker copy per ingest and a snapshot request on subscribe or recovery. Co-location could reduce fanout but would tie subscription placement and scaling to resort partitions.

RabbitMQ resource	Binding and purpose
skitrak.exchange	Durable topic with routing key resort.{resortID}
resort.1 through resort.10	One durable queue per resort. TTL 600,000 ms, length limit 1,000,000, reject-publish overflow.
skitrak.leaderboard.updates	Topic bound to a dedicated, exclusive queue for each ingest. Versions are keyed by resort and day.
skitrak.leaderboard.snapshots	Correlated request/reply with a 5-second request TTL, queue limit of 1,000 and exclusive reply queues.
skitrak.dlx / skitrak.deadletter.queue	Dead-letter path for expired or rejected swipes under the deployed policy.

The independent EC2 hosts separate the four Tomcat ingests, RabbitMQ, and the processing Consumer. The load generator shared the RabbitMQ host for the runs in this report.

Swipe to leaderboard



Publishing and ordered aggregation

The servlet validates input and uses the fixed lift vertical table. It borrows a channel exclusively from a pool on one publishing connection and returns it after confirm. Connection recovery rebuilds invalid channels. The Consumer gives each resort one serial admission path on a configurable worker pool. A round-robin batcher selects from the resort queues and preserves FIFO within each resort.

The in-memory store applies event ID deduplication, vertical totals, the top 10 and pending versioned updates under one lock. Manual ACK advances only over the completed delivery-tag prefix. The dispatcher publishes pending versions independently and retries failed publications. Ingest hosts ignore duplicate or older versions and resynchronize subscriptions after broker recovery. A failed aggregation batch retries ahead of later batches.

ALB and failure handling

The deployment script specifies ALB cookie stickiness, a 120-second idle timeout, and /server-v2/health at 30-second intervals with a 5-second timeout and thresholds of 2 healthy / 3 unhealthy. Tomcat accepts the WebSocket upgrade. Ingest readiness checks the backplane and systemd restarts services. Observed failover and configuration limits are on page 7.

Memory state and deduplication last for one process lifetime. An ACKed in-memory swipe is lost from the standings after a Consumer restart. SQLite is a separate startup mode with durable aggregation and a transactional outbox. The worker count is chosen at startup; automatic worker scaling is unimplemented.

AWS test matrix and client output

November 13, 2026 · PST · Separate load windows for each configuration

Workload	Request path	HTTP test window	Client console
1 ingest / 500K	direct VPC	15:58:15 to 16:03:11	Full log
2 ingests / 500K	ALB	16:08:48 to 16:13:21	Full log
4 ingests / 500K	ALB	16:18:58 to 16:23:32	Full log
4 ingests / 1M	ALB	16:28:05 to 16:35:28	Full log

Each run subscribed to all ten resorts on a fresh test day. The client used 32,000 warmup swipes at 300/s with 32 threads, followed by an unpaced main phase with 128 threads. The main phase sent 468,000 swipes in each 500K run and 968,000 in the 1M run.

Client console results

Metric	1 / 500K	2 / 500K	4 / 500K	4 / 1M
HTTP runtime incl. warmup (s)	295.83	272.90	274.01	442.82
HTTP overall throughput (req/s)	1,690.15	1,832.15	1,824.74	2,258.27
HTTP main throughput (req/s)	2,478.33	2,824.75	2,805.71	2,883.09
HTTP mean (ms)	47.97	42.42	42.77	42.84
HTTP P95 (ms)	84.45	68.17	69.91	68.31
HTTP P99 (ms)	115.42	90.74	92.08	90.44
Successful swipes	500,000	500,000	500,000	1,000,000
Failed swipes	0	0	0	0
Logged retry triggers	0	0	0	0
Distinct WebSocket updates	1,557	1,636	1,646	1,944

Main-phase throughput rose 14.0% from direct single ingest to two ingests through the ALB. At 500K, four ingests came within 0.7% of two and both ALB runs had lower HTTP P99 than the direct run. The route changed from direct VPC to ALB at the same time as the ingest count, so these observations combine both effects.

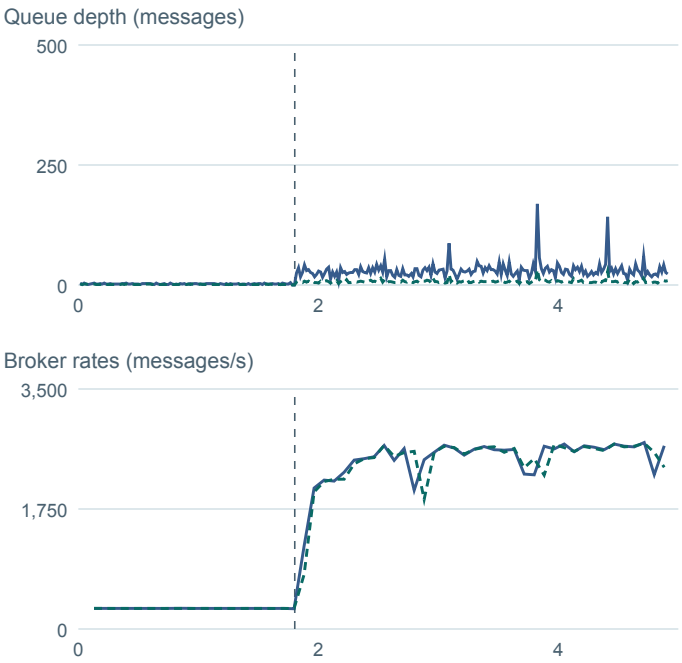
The four-ingest 1M run sustained 2.8% higher main-phase throughput than its 500K run. Consumer P99 rose from 60 to 74 ms while HTTP P99 moved from 92.08 to 90.44 ms.

Each client logged validation passed. Across all four runs, per-day HTTP 201, broker publish/deliver/ACK and unique Consumer totals matched. All ten business queues drained, with no new dead letters or broker redeliveries.

RabbitMQ queue depth and message rates

Blue-gray = total depth or publishes. Teal dashes = largest single resort or consumer ACKs. Time is minutes from first swipe. Vertical dashes mark the start of the unpaced phase.

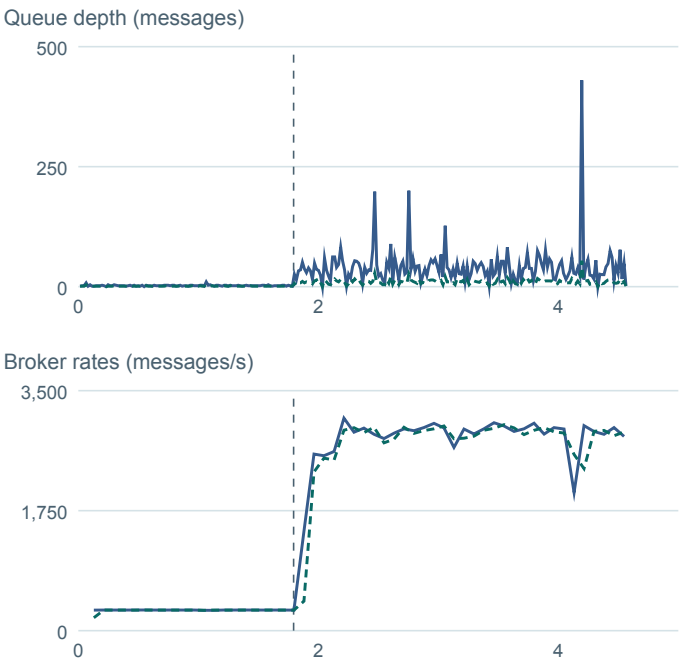
1 ingest / 500K



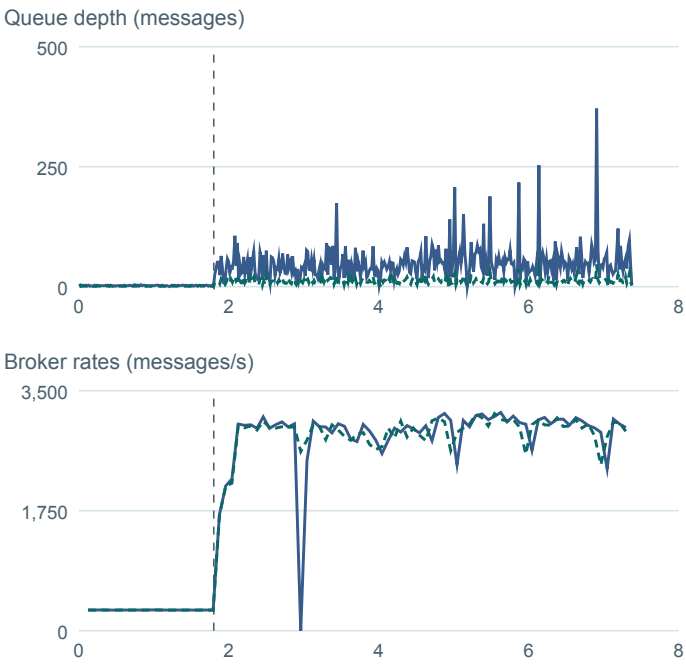
2 ingests / 500K



4 ingests / 500K



4 ingests / 1M



Queue depth is sampled Ready + Unacked, approximately every second. Broker rates are from approximately five-second counter intervals. Queue charts exclude the 84,382 historical dead letters.

Queue targets and EC2 resources

The queue target is below 1,000 messages. The Consumer lag target is below 100 ms.

Metric	1 / 500K	2 / 500K	4 / 500K	4 / 1M
Consumer mean (ms)	33.88	19.14	16.28	17.25
Consumer P99 (ms)	102	70	60	74
Consumer max (ms)	687	380	339	204
Swipes with Consumer lag ≥100 ms	5,534	1,015	611	2,631
Largest resort queue depth	32	35	54	46
Aggregate queue depth, mean	20.02	27.41	26.05	37.69
Aggregate queue depth, max	169	255	430	371
Mean publish (msg/s), sampled interval	1,716.4	1,837.9	1,870.3	2,274.5
Mean ACK (msg/s), sampled interval	1,703.3	1,808.9	1,830.2	2,284.9

At the sampled peaks, every business queue and their aggregate stayed below 1,000. Consumer lag measures ingest event creation to aggregation start. The fractions at or above 100 ms were 1.107%, 0.203%, 0.122% and 0.263%, respectively. The target was exceeded in the tail of each run.

Run	EC2 role	Main CPU	Peak service MiB	RX MiB	TX MiB
1 / 500K	ingest-1	34.9%	270.2	236.8	372.2
	consumer	18.3%	341.8	202.7	23.9
	broker + client	99.3%	165.8	395.7	443.7
2 / 500K	ingest-1	20.0%	250.8	149.2	174.6
	ingest-2	21.2%	237.0	149.2	174.6
	consumer	18.2%	381.5	196.1	19.2
	broker + client	99.2%	182.2	585.3	453.5
4 / 500K	ingest-1	10.8%	228.1	78.3	90.6
	ingest-2	10.4%	215.6	78.2	90.6
	ingest-3	15.6%	191.5	78.4	90.5
	ingest-4	13.2%	190.6	78.4	90.4
	consumer	16.5%	380.6	199.8	18.6
	broker + client	99.4%	200.4	601.9	467.7
4 / 1M	ingest-1	10.7%	237.8	155.2	180.7
	ingest-2	10.7%	225.5	155.2	180.8
	ingest-3	14.3%	199.7	155.5	180.5
	ingest-4	12.5%	198.8	155.5	180.6
	consumer	18.2%	597.0	398.4	33.7
	broker + client	98.9%	197.3	1,201.9	927.5

CPU is host busy time during the main phase. Memory is peak monitored service memory. RX/TX are host network deltas within the HTTP test window. The broker/client host was 98.9–99.4% busy while the Consumer stayed at 16.5–18.3%. Shared host CPU is a likely constraint. These samples cannot distinguish load generator CPU from broker CPU.

Load distribution and live leaderboard proof

Successful POSTs	Ingest 1	Ingest 2	Ingest 3	Ingest 4
1 ingest / 500K	500,000	—	—	—
2 ingests / 500K	250,000	250,000	—	—
4 ingests / 500K	125,000	124,999	125,001	125,000
4 ingests / 1M	249,998	250,000	249,999	250,003

The table uses day-scoped ingest access logs. Each of two targets handled 50%, and each of four targets handled about 25%. CloudWatch shows the average request count per healthy target. Its **Sum** statistic therefore complements the individual access-log totals. AWS metric definition.

Real swipe, real standings

Controlled memory-mode probe	Observed value
Event ID on both POST and update	f76b8bf9-d852-4ce1-861d-9c76e42c2be9
Swipe and snapshot	Lift 6, 397 ft. Resort 1, skier 98001, rank 1.
Version and replay	Version 1 arrived over WebSocket. Replaying the ID left the total and version unchanged.

The four loads received 1,557, 1,636, 1,646 and 1,944 distinct updates, respectively, matching their final version sums. Each ALB run returned identical final ordered standings and versions across the participating ingests. The two-target probe returned 6/6 cookie replays to the same target. Each four-target probe returned 12/12. Controlled swipe JSON.

Idle and failover validation

In a separate Nov 9 probe on the earlier SQLite deployment, an idle TCP connection remained reusable after 75 seconds. After stopping one ingest, its WebSocket closed. A reconnect with the original cookie reached a different healthy target after 50.62 seconds and received a controlled real update. Twelve health probes returned HTTP 502 during detection. The restored ingest re-entered ALB routing 147.05 seconds after local readiness.

The failover probe sent one swipe after reconnect. Failover under continuous load and a live export of the exact idle timeout and health thresholds were outside the measured evidence. Failover probe results.

Tuning and interpretation

Concurrency and prefetch

Short local memory-mode runs used four ingests and 30,000 swipes. The client sweep kept 10 workers. The worker sweep kept 128 client threads. The prefetch comparison used 128 client threads and 10 workers. Reported throughputs are local and cannot estimate AWS capacity.

Local setting	HTTP req/s	HTTP P99 ms	Consumer P99 ms
Client 64	6,324	39.1	30
Client 128 / workers 10	8,846	48.6	38
Client 256	8,053	126.8	84
Client 512	8,017	244.4	183
Workers 20	9,109	51.8	24
Workers 40	9,284	49.6	40
Workers 80	9,299	46.8	109
Prefetch 25	8,435	56.8	29

In the client sweep, 128 threads had the highest throughput. Moving from 40 to 80 workers scarcely changed throughput and raised Consumer P99 from 40 to 109 ms. The AWS runs used 40 workers, prefetch 50, a batch cap of 50 with a 5 ms wait, and an ACK cap of 25 with a 5 ms flush. Publisher channel pool size was held at 32 in the documented tuning configuration. Pool size and ACK batch size were held fixed in the main AWS suite. Local tuning data.

Storage diagnosis

An isolated local broker test sent the same workload at 50,000 events/s. Median ACK throughput including drain was 28,254/s with original SQLite, 37,581/s with optimized SQLite and 49,774/s with memory. Decreasing the storage work reduced backlog in this controlled case. This short batch test measured publish-to-ACK latency; the AWS tests measure event creation to aggregation start. Storage A/B data.

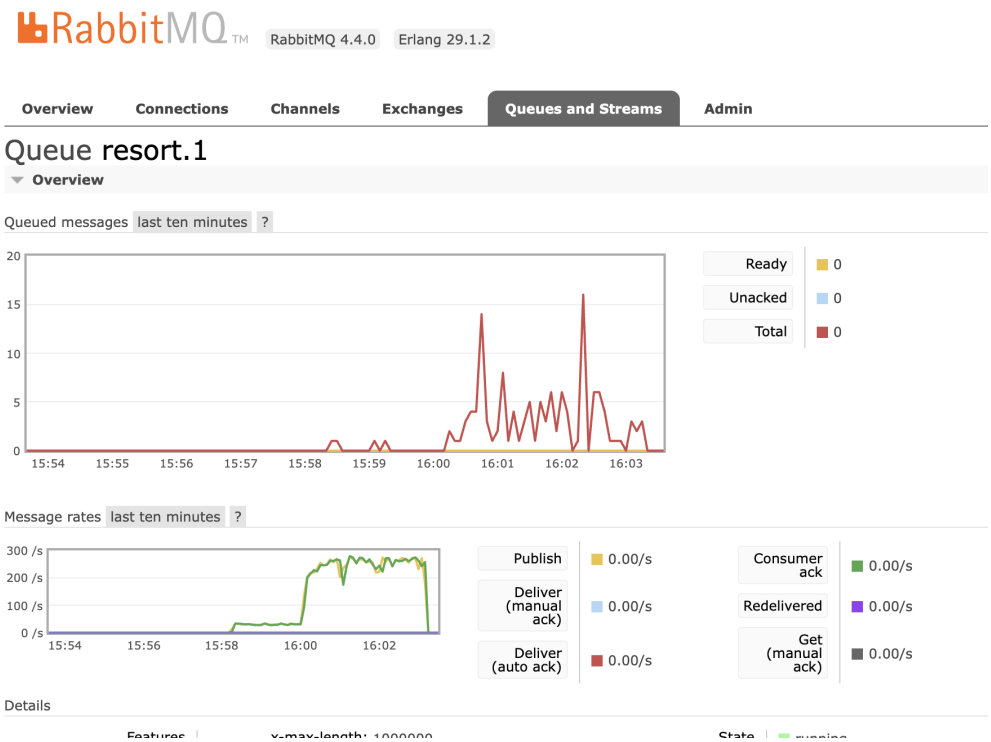
Recovery from the earlier interruption

A prior single-ingest public-path run stopped after 39,965 HTTP 201s and matching ACKs. The guard stopped new requests when queue samples became stale. The client logged 78 connection timeouts and the highest sampled queue depth was five. We moved the client and guard into the AWS VPC, then completed the 500K repeat. The precise network failure point was unconfirmed.

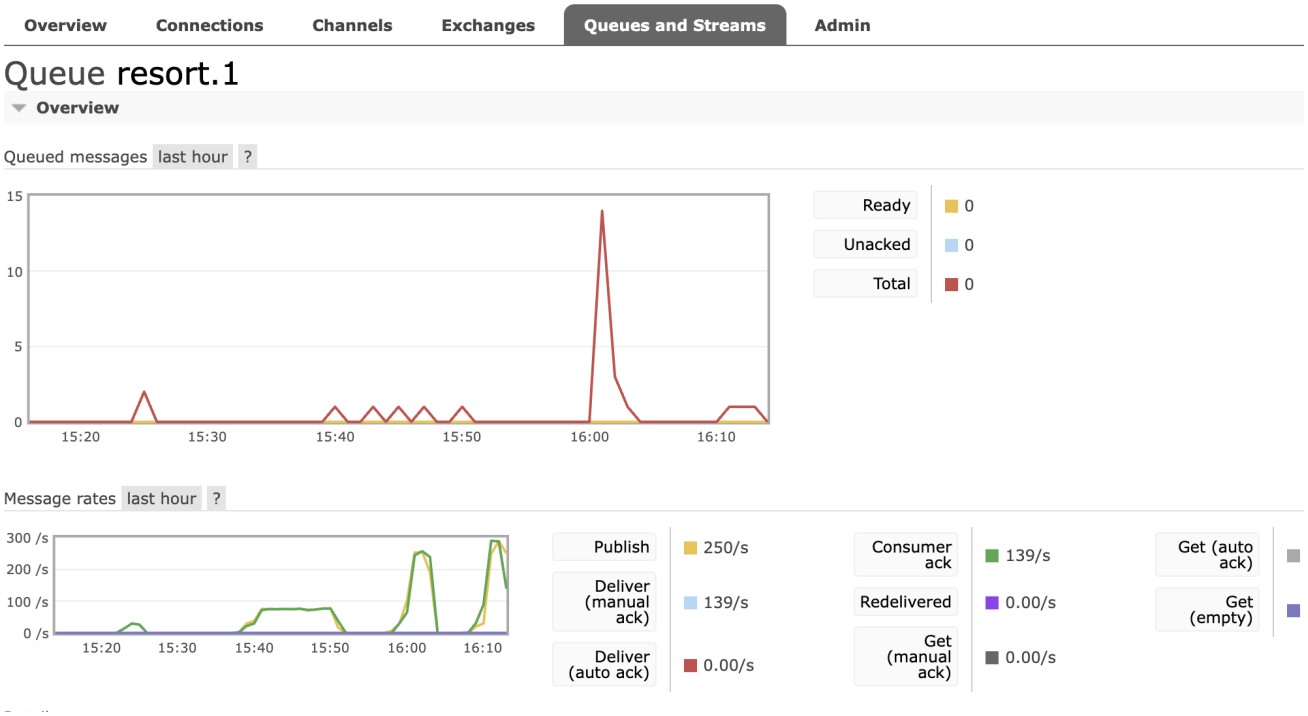
All in-memory AWS runs used the temporary memory deployment. The original SQLite service was restored after collection; its JAR and the 7,507,212 processed-event count matched the pretest values.

RabbitMQ console evidence

500K single- and two-ingest runs. The console shows the resort.1 queue. The full-run sampler on pages 5–6 covers all ten business queues.



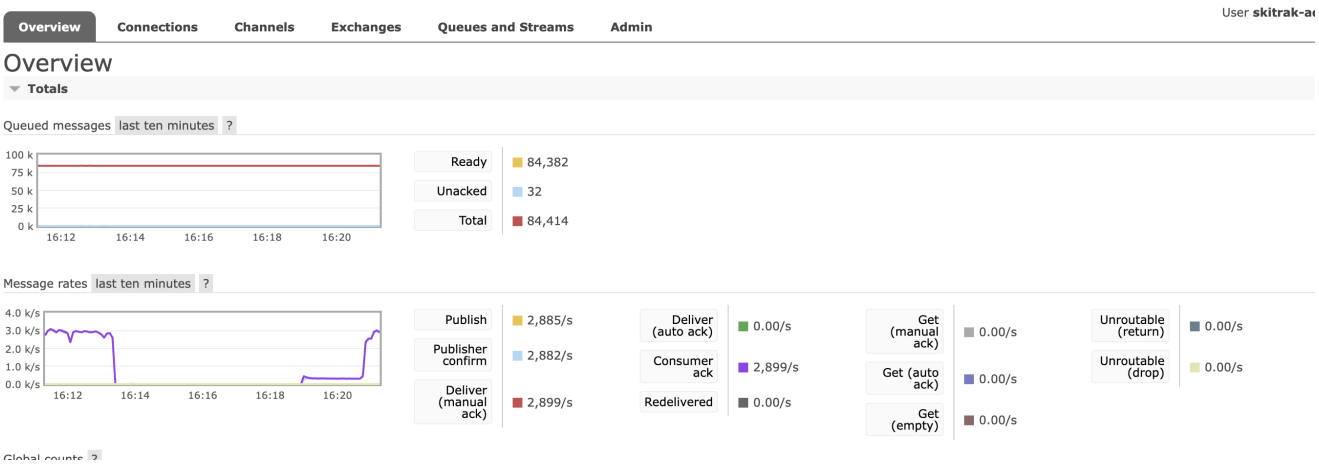
Single-ingest 500K. The 16:03:36 capture shows warmup, the unpaced main phase and the drained queue. Full-size capture



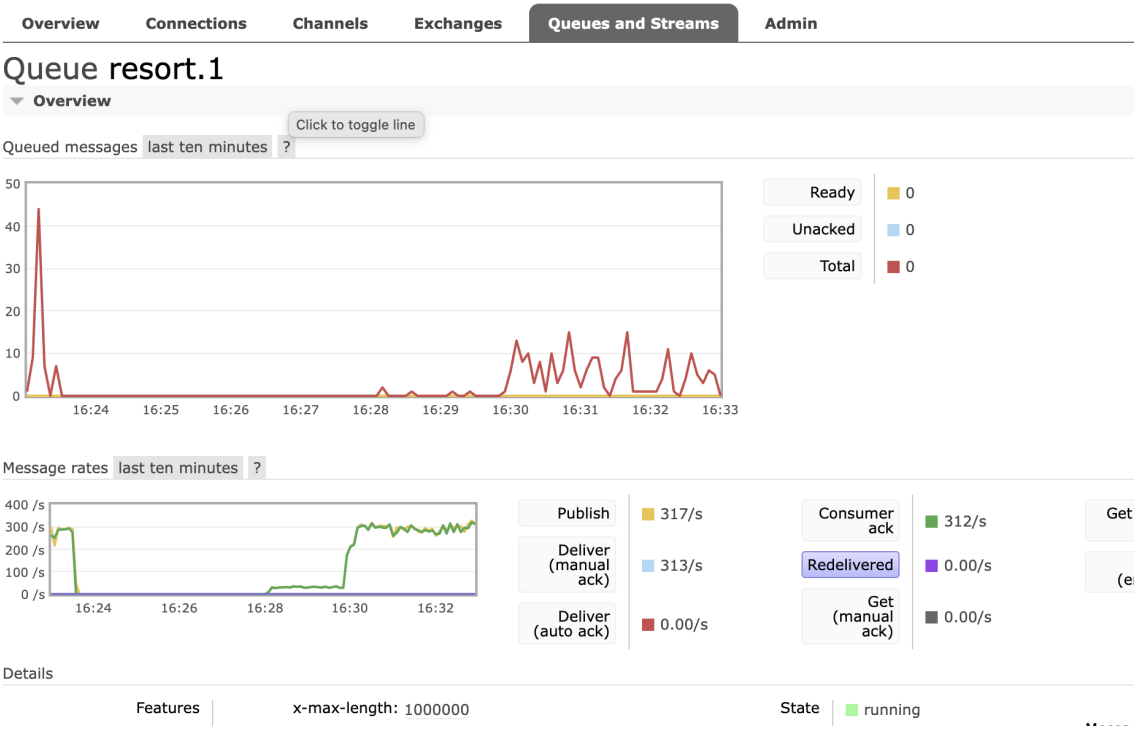
Two-ingest 500K. The rightmost burst is the test run. The preceding burst belongs to the single-ingest run. Full-size capture

RabbitMQ console evidence

500K and 1M four-ingest runs. Each capture time is in PST.



Four-ingest 500K. At 16:21:25 during the main phase, broker Overview showed about 2,885 publishes/s and 2,899 ACKs/s. Ready included the existing 84,382 dead letters. Full-size capture

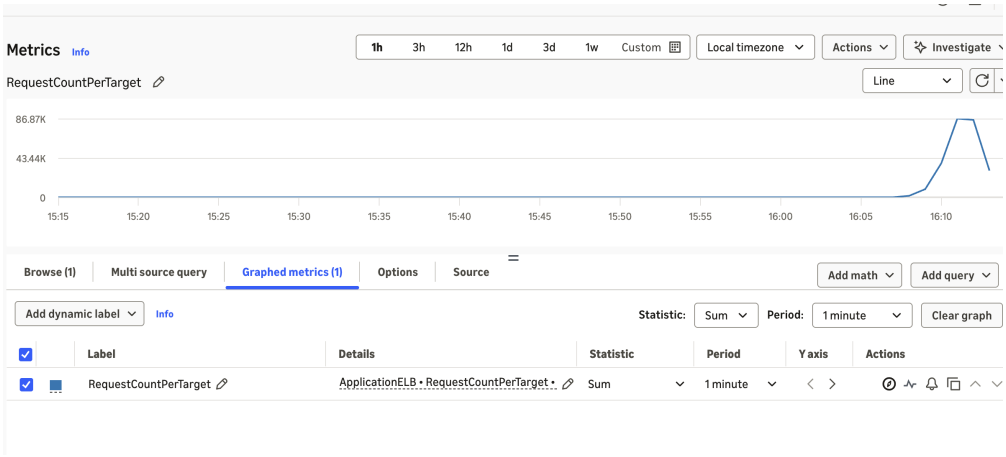


Four-ingest 1M. At 16:33:04 the resort.1 queue was near empty while publish and ACK rates tracked. The earlier leftmost spike predates this run. Full-size capture

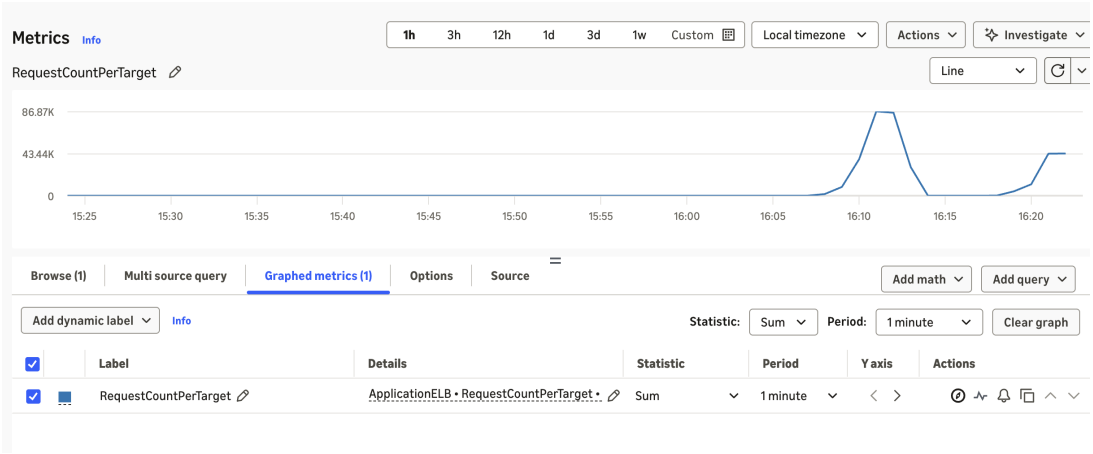
The four-ingest 500K resort.1 capture shows warmup. The 1M broker Overview shows about 3,033 publishes/s and 3,052 ACKs/s during the main phase.

CloudWatch ALB evidence

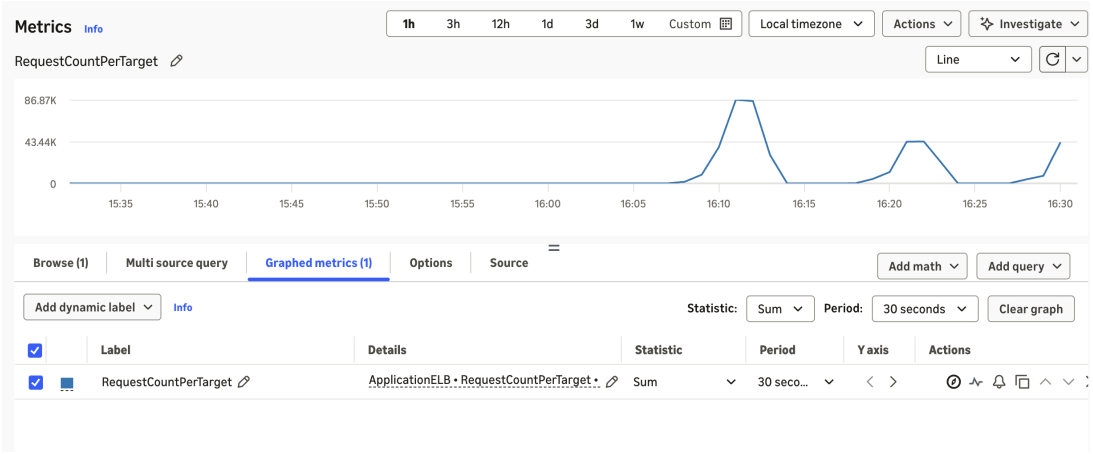
RequestCountPerTarget across the healthy targets. The exact per-ingest POST counts are on page 7.



Two ingests / 500K, 16:15:17. The rightmost peak covers the two-ingest run, with Sum and a 1-minute period. Full-size capture



Four ingests / 500K, 16:24:30. The later burst shows about 43K requests per target per minute. The earlier burst belongs to the two-ingest run. Full-size capture



Four ingests / 1M, 16:32:34. Only the rightmost rise belongs to this still-running test. The console shows Sum and a 30-second period. Full-size capture

Client console excerpts

The final HTTP summary and two WebSocket validation lines from each run.

1 ingest / 500K · Full client output

```
Successful Swipes: 500000
Failed Swipes: 0
HTTP Runtime: 295.83 seconds
Overall HTTP Throughput: 1690.15 req/sec
Distinct standings changes received: 1557
Invalid leaderboard payloads: 0
```

2 ingests / 500K · Full client output

```
Successful Swipes: 500000
Failed Swipes: 0
HTTP Runtime: 272.90 seconds
Overall HTTP Throughput: 1832.15 req/sec
Distinct standings changes received: 1636
Invalid leaderboard payloads: 0
```

4 ingests / 500K · Full client output

```
Successful Swipes: 500000
Failed Swipes: 0
HTTP Runtime: 274.01 seconds
Overall HTTP Throughput: 1824.74 req/sec
Distinct standings changes received: 1646
Invalid leaderboard payloads: 0
```

4 ingests / 1M · Full client output

```
Successful Swipes: 1000000
Failed Swipes: 0
HTTP Runtime: 442.82 seconds
Overall HTTP Throughput: 2258.27 req/sec
Distinct standings changes received: 1944
Invalid leaderboard payloads: 0
```